

BUILDING LARGE LANGUAGE MODELS FROM SCRATCH

A Comprehensive Guide from Fundamentals to Production

Based on the YouTube Playlist by Andrej Karpathy et al.

Version 1.0

Table of Contents

Foreword

Chapter 1: Introduction to Language Models and Tokenization

Chapter 2: Understanding Embeddings and Vector Spaces

Chapter 3: The Attention Mechanism Explained

Chapter 4: Building the Transformer Architecture

Chapter 5: Pre-training and the Role of Data

Chapter 6: Fine-tuning and Instruction Following

Glossary

Foreword

The study of large language models sits at a fascinating intersection of mathematics, computer science, and linguistics. What began as a modest idea in information theory — that sequences of words follow statistical patterns — has grown into one of the most consequential technological developments of our time. Yet despite their ubiquity, the internal workings of these systems remain opaque to many practitioners who use them daily.

This book exists to change that. Written from the ground up, it takes the reader from the most elementary operations — how raw text is broken into tokens — through the intricate dance of self-attention, all the way to the training dynamics that breathe coherence into billions of parameters. Each chapter builds strictly on what came before, so there are no conceptual gaps left to leap across.

The material presented here is derived from a hands-on video series that has introduced thousands of engineers to the craft of building language models. The videos walk through working code, make mistakes in real time, and explain not just what the code does but why each design decision was made. This written form preserves that spirit of careful reasoning while adding the density and reference-ability that prose affords over video.

A word on prerequisites: the reader is assumed to be comfortable with Python and to have some intuition about vectors and matrix multiplication. Deep familiarity with machine learning is helpful but not required — every new concept is introduced with the motivation before the mechanics. The goal is not to produce a catalog of facts but to build genuine understanding that allows the reader to extend, modify, and question these systems independently.

Chapter 1

Introduction to Language Models and Tokenization

A language model is, at its most fundamental level, a system that assigns probabilities to sequences of text. Given the phrase "the cat sat on the", a language model should assign a higher probability to "mat" or "floor" as the next word than to "algorithm" or "democracy." This seemingly simple task — predicting what comes next — turns out to encode an enormous amount of knowledge about grammar, facts, reasoning, and even social convention.

Before a language model can process text, it must convert that text into a format amenable to mathematical operations. Raw characters are not directly useful; instead, the text is divided into discrete units called tokens. A token might correspond to a whole word, a common subword fragment, or sometimes a single character, depending on the tokenization scheme chosen.

Why Tokenization Matters

The choice of tokenization scheme has surprisingly deep consequences for model behavior. Consider the word "unhappiness": a character-level tokenizer would split this into 11 separate units, each individually carrying little semantic weight. A word-level tokenizer would treat it as a single unit, but would struggle with rare words, misspellings, and languages with complex morphology. Subword tokenization, which lies between these extremes, splits "unhappiness" into fragments like "un", "happiness" — preserving the meaningful components while keeping the vocabulary size manageable.

The most common algorithm for subword tokenization today is Byte Pair Encoding, or BPE. The algorithm begins with a vocabulary of individual characters and iteratively merges the most frequently co-occurring pair of symbols. Repeated over thousands of iterations, this process produces a vocabulary that contains common words as single tokens while representing rarer words through combinations of frequent subword fragments. The vocabulary size is a hyperparameter: typical values range from around 32,000 to 100,000 tokens.

Building a Tokenizer from Scratch

To truly understand tokenization, it helps to build a minimal BPE tokenizer by hand. The process begins by reading a corpus of text and computing the frequency of every consecutive pair of characters or tokens. The most frequent pair is identified — suppose it is the pair ("t", "h"), appearing 14,000 times — and merged into a single new token "th". This merged token is then added to the vocabulary, and all occurrences of the pair in the corpus are replaced with the new token.

The process repeats: in the next iteration, the most frequent pair might be ("th", "e"), yielding the token "the". By the time the algorithm has performed 50,000 merges, the vocabulary contains tokens ranging from single characters to common words and phrases. The merge rules are saved alongside the vocabulary; at inference time, a new piece of text is tokenized by applying the same merge rules in the same order.

One subtlety that practitioners often overlook is how special tokens are handled. Tokens like `<|endoftext|>` are inserted to mark document boundaries, and the model must learn during training that these tokens signal a transition between independent documents rather than a semantic continuation. Getting this right has real implications for how well the model learns to reset context at document boundaries.

Chapter 2

Understanding Embeddings and Vector Spaces

Once text has been reduced to a sequence of integer token IDs, the next challenge is to represent those integers in a way that captures semantic relationships. The raw integer 4,821 carries no inherent meaning — it is simply an index into a vocabulary table. What transforms it into something mathematically useful is embedding: the mapping of each token ID to a dense vector of real numbers.

An embedding table is a matrix of shape `[vocabulary_size, embedding_dimension]`. When the model processes token ID 4,821, it performs a simple table lookup — retrieving the row at index 4,821 — to obtain a vector of, say, 768 floating-point numbers. This vector is the token's representation within the model's internal geometry.

The Geometry of Meaning

The remarkable property of well-trained embeddings is that semantic similarity corresponds to geometric proximity. Vectors for words like "king" and "queen" lie close together in the embedding space, while the vector for "algorithm" lies far away from both. Even more striking are the arithmetic relationships that emerge: the vector for "king" minus the vector for "man" plus the vector for "woman" lies very close to the vector for "queen."

These relationships are not programmed; they arise from the statistical structure of language. Words that appear in similar contexts develop similar vectors because the training process adjusts vectors to predict context accurately. The embedding space becomes, in effect, a compressed map of semantic relationships extracted from the training corpus.

Positional Encoding

Token embeddings capture what a token means but not where it appears in a sequence. The word "not" means the same thing whether it appears at position 3 or position 23 in a sentence, yet its position is crucial to interpreting the sentence's meaning. To give the model positional awareness, a positional encoding vector is added to each token embedding before the sequence is processed.

The original Transformer architecture used fixed sinusoidal positional encodings, where each position maps to a unique combination of sine and cosine waves at different frequencies. More recent architectures use learned positional embeddings, where the positional vectors are treated as parameters trained alongside the rest of the model. Still newer approaches, such as Rotary Position Embedding (RoPE), encode position through rotations applied directly to the attention computation rather than through additive offsets.

Chapter 3

The Attention Mechanism Explained

The attention mechanism is the conceptual heart of the Transformer architecture and, by extension, of every modern large language model. Understanding it deeply — not just operationally but geometrically and intuitively — is essential for anyone who wants to reason about model behavior, diagnose failure modes, or design improvements.

The central intuition is this: when processing any given token, the model should be able to look back at other tokens in the sequence and decide how relevant each one is to understanding the current token's meaning. The word "it" in a long sentence, for example, needs to resolve to some earlier noun; attention provides the mechanism by which the model can look back, identify the most likely referent, and incorporate information from that referent when building the representation of "it."

Queries, Keys, and Values

Attention is parameterized through three learned linear projections: the Query projection, the Key projection, and the Value projection. For each token in the sequence, three vectors are computed: a query vector Q representing "what this token is looking for," a key vector K representing "what this token contains," and a value vector V representing "the information this token will contribute if selected."

Attention scores between a query token and all key tokens are computed as dot products: a high dot product between query i and key j means token i finds token j highly relevant. These raw scores are scaled by the square root of the key dimension to prevent the dot products from growing too large in magnitude, then passed through a softmax to produce a probability distribution over all positions. The output for token i is then the weighted sum of all value vectors, with weights given by the attention distribution.

The scaling factor deserves more than a footnote. Without it, as embedding dimensions grow large, dot products between random vectors tend to have large variance, causing the softmax to saturate: one score dominates and receives nearly all the weight while others receive essentially zero. This kills gradient flow. The square-root scaling restores the variance to a manageable range.

Multi-Head Attention

A single attention computation can only capture one kind of relationship at a time. Multi-head attention addresses this by running several attention operations in parallel, each with its own learned Q, K, and V projections but operating on a lower-dimensional subspace. The outputs of all heads are concatenated and projected back to the full model dimension.

Different heads learn to specialize in different relationship types. One head might track syntactic subject-verb agreement; another might follow coreference chains; a third might model positional proximity. This specialization is not designed in — it emerges from training on the prediction task, as different patterns of co-occurrence require different kinds of contextual aggregation.

Chapter 4

Building the Transformer Architecture

With tokenization, embeddings, and attention in hand, the pieces are in place to assemble a complete Transformer. The architecture is built by stacking identical blocks, each consisting of a multi-head self-attention layer followed by a position-wise feed-forward network, with residual connections and layer normalization around each sub-layer.

A Transformer block takes a sequence of vectors as input and produces a sequence of vectors of the same shape as output. The residual connection means that the block's output is added to its input, so each block is only required to learn a residual update rather than the full transformation. This design choice dramatically improves gradient flow in deep networks, making it practical to stack 96 or more layers.

Layer Normalization

Layer normalization is applied before each sub-layer in the modern "pre-norm" variant of the Transformer. It normalizes each token's vector to have zero mean and unit variance, then applies learned scale and shift parameters. Unlike batch normalization, which normalizes across the batch dimension, layer normalization operates independently on each sequence position, making it compatible with variable-length sequences and small batch sizes.

The placement of normalization has subtle effects on training dynamics. The original Transformer used "post-norm" (normalization after the residual addition), which requires careful learning rate warmup to avoid instability in the early stages of training. Pre-norm Transformers are more stable and have become the standard for large-scale training, at the cost of a slight degradation in final performance that is generally outweighed by the training stability gains.

The Feed-Forward Network

Each Transformer block contains a two-layer feed-forward network applied independently to each position. The first layer expands the dimension by a factor of four (so a model with dimension 768 expands to 3,072), applies a non-linear activation function, and the second layer projects back to the original dimension. The expansion and contraction pattern gives the network capacity to mix information across the feature dimension in ways that attention alone cannot.

The choice of activation function matters more than it might appear. The original Transformer used ReLU. More recent models use GELU (Gaussian Error Linear Unit), which is smoother than ReLU and has been empirically found to improve performance. The newest architectures often use SwiGLU, a gated variant that introduces a learned gating mechanism inside the feed-forward block, further increasing expressiveness at modest cost.

Chapter 5

Pre-training and the Role of Data

A Transformer architecture is, by itself, a randomly initialized function mapping token sequences to logits. What transforms it into a system with apparent knowledge, reasoning ability, and linguistic fluency is pre-training: the process of updating billions of parameters by repeatedly exposing the model to text and adjusting the parameters to reduce the prediction error.

The pre-training objective for autoregressive language models is next-token prediction. For every position in every training sequence, the model predicts a probability distribution over the vocabulary for the next token, and the parameters are updated via stochastic gradient descent to increase the probability assigned to the actual next token. Summed across positions, sequences, and billions of training steps, this simple objective produces a model that has internalized an enormous amount of knowledge and structure from the training corpus.

Data Quality and Scale

The composition of the training corpus has a first-order effect on model behavior. A model trained primarily on scientific papers will excel at reasoning about scientific topics but may struggle with casual conversation. A model trained on web data will have broad coverage but will also have absorbed the biases, factual errors, and rhetorical patterns prevalent on the internet.

Modern pre-training datasets are assembled from multiple sources: web crawls, books, code repositories, academic papers, and curated high-quality text. The mixing ratios are themselves hyperparameters, tuned based on the desired capability profile of the final model. Overweighting code produces models better at programming tasks; overweighting instruction-following data produces models that generalize better to novel tasks.

Data filtering is at least as important as data collection. Common filtering steps include deduplication (removing near-identical documents that would cause the model to memorize rather than generalize), quality filtering (removing low-coherence text, spam, and automatically generated content), and safety filtering (removing content that would cause the model to learn harmful patterns).

Chapter 6

Fine-tuning and Instruction Following

A language model trained on next-token prediction is a powerful statistical engine, but it is not yet a useful assistant. Given the prompt "What is the capital of France?", a raw pre-trained model is likely to continue the text in the style of a document that contains such questions — perhaps generating another question rather than answering the one posed. Fine-tuning bridges the gap between raw language modeling capability and the aligned, helpful behavior expected of a deployed assistant.

The first stage of fine-tuning is supervised fine-tuning on instruction-following demonstrations. A dataset of (instruction, response) pairs is assembled, often through a combination of human curation and model-assisted generation. The model is trained to predict the response tokens given the instruction, using the same next-token prediction objective as pre-training but on this targeted data.

Reinforcement Learning from Human Feedback

Supervised fine-tuning improves instruction following but has a fundamental limitation: the model is trained to imitate the demonstrations, and the quality of the resulting behavior is bounded by the quality of those demonstrations. A more powerful approach is to train a reward model that predicts human preference between two candidate responses and then use reinforcement learning to optimize the policy toward responses that the reward model scores highly.

This process, known as Reinforcement Learning from Human Feedback (RLHF), has been central to the development of aligned language models. Human annotators are shown pairs of model responses to the same prompt and asked to indicate which they prefer. These preference judgments are used to train a scalar reward model, and the language model is then updated using policy gradient methods to maximize the reward model's scores.

The primary risk in RLHF is reward hacking: the language model may find ways to produce responses that score highly according to the reward model without actually being better responses. Mitigation strategies include regularization toward the supervised fine-tuning baseline (penalizing large deviations from the pre-RLHF policy), careful design of the reward model training data, and iterative refinement of the feedback collection process.

Glossary

Attention Mechanism: A neural network operation that allows each position in a sequence to incorporate information from other positions, weighted by learned relevance scores derived from query-key dot products.

BPE (Byte Pair Encoding): A tokenization algorithm that iteratively merges the most frequent consecutive symbol pairs in a corpus to construct a subword vocabulary.

Embedding: A dense vector representation of a discrete token, stored as a row in a learnable embedding matrix indexed by token ID.

Feed-Forward Network: The position-wise two-layer MLP within each Transformer block, typically expanding the hidden dimension by a factor of four before projecting back.

GELU (Gaussian Error Linear Unit): A smooth activation function that multiplies its input by the cumulative distribution function of the standard normal, commonly used in Transformer feed-forward networks.

Layer Normalization: A normalization technique that standardizes each token's feature vector independently, stabilizing training in deep networks.

Multi-Head Attention: An extension of single-head attention that runs multiple attention operations in parallel on lower-dimensional projections, allowing the model to capture different relationship types simultaneously.

Next-Token Prediction: The pre-training objective of autoregressive language models, where the model is trained to assign high probability to the actual next token given all preceding tokens.

Positional Encoding: A representation added to token embeddings to inject information about each token's position within the sequence.

Pre-training: The large-scale unsupervised training phase where model parameters are learned from a broad text corpus via next-token prediction.

Residual Connection: An architectural pattern where a layer's input is added to its output, allowing gradients to flow directly through deep networks and enabling each layer to learn an incremental update.

RLHF (Reinforcement Learning from Human Feedback): A fine-tuning methodology that uses human preference judgments to train a reward model, then applies reinforcement

learning to align the language model toward preferred responses.

Softmax: A function that converts a vector of arbitrary real values into a probability distribution by exponentiating each element and normalizing by the sum.

Subword Tokenization: A tokenization strategy, including BPE and WordPiece, that represents text as a mixture of whole words and word fragments, balancing vocabulary coverage against vocabulary size.

Supervised Fine-tuning: Training a pre-trained model on curated (instruction, response) pairs to improve instruction-following behavior.

Token: The basic unit of text used by a language model, typically corresponding to a word, subword fragment, or character depending on the tokenization scheme.

Transformer: A neural network architecture based on self-attention and feed-forward layers, introduced in "Attention is All You Need," that forms the basis of all modern large language models.

Vocabulary: The finite set of tokens a model can process and generate, constructed during tokenizer training and fixed at model training time.